

Rapport DevOps — Mini Helpdesk API

Module: DevOps | Année universitaire: 2025-2026

Sujet: Plateforme de tickets de support (mini Helpdesk)

Étudiants: Mouaad Chakir & Souad Chakir

Dépôt GitHub: <https://github.com/mouaadchakir/devops-helpdesk-api>

Ce rapport présente la réalisation complète du projet: application métier, stratégie Git, conteneurisation Docker, pipeline CI/CD et déploiement Kubernetes (bonus).

Livrables

- Code source versionné avec workflow par branches
- API REST sécurisée et documentée (Swagger/OpenAPI)
- Image Docker et orchestration Docker Compose
- Pipeline CI/CD GitHub Actions (build/test/publish)
- Manifestes Kubernetes (bonus)

1. Introduction et objectifs

Le projet consiste à implémenter une API Helpdesk permettant la création et le suivi des tickets support. Le système gère les utilisateurs, les rôles, les réponses agents et le cycle de vie des tickets.

L'objectif DevOps est de garantir un flux de développement professionnel: versionnement clair, automatisation des tests, packaging conteneurisé et préparation au déploiement.

2. Architecture et choix techniques

La solution est basée sur Spring Boot 3.3.x (Java 17), Spring Data JPA, Spring Security, MySQL et OpenAPI. Les couches Controller/Service/Repository structurent la logique métier.

Le modèle de données comprend les entités AppUser, Role, Ticket et TicketResponse. Les rôles ROLE_USER, ROLE_AGENT et ROLE_ADMIN appliquent une gouvernance des accès adaptée au Helpdesk.

3. Processus DevOps

Le dépôt suit une stratégie de branches main/develop/feature avec Pull Requests. Cette stratégie améliore la traçabilité des changements et réduit le risque d'intégration.

Le pipeline GitHub Actions exécute les tests Maven, construit l'artefact applicatif, puis construit/publie l'image Docker sur GHCR lors des pushes sur la branche main.

4. Conteneurisation et déploiement

La conteneurisation repose sur un Dockerfile multi-stage pour optimiser la taille finale de l'image. Le lancement local est simplifié via docker-compose.

En bonus, un dossier k8s/ fournit les manifestes ConfigMap, Secret, Deployments et Services afin de déployer l'application sur cluster Kubernetes local.

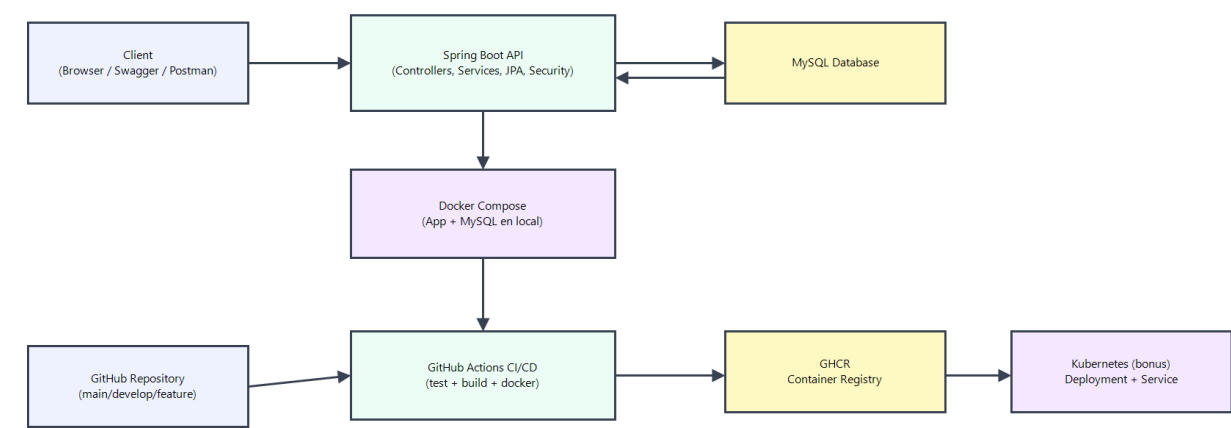
5. Preuves par captures d'écran

Les captures suivantes servent d'évidence de fonctionnement de l'application, du pipeline CI/CD et du déploiement Kubernetes.

Capture 1: Architecture globale du projet

01_architecture.png

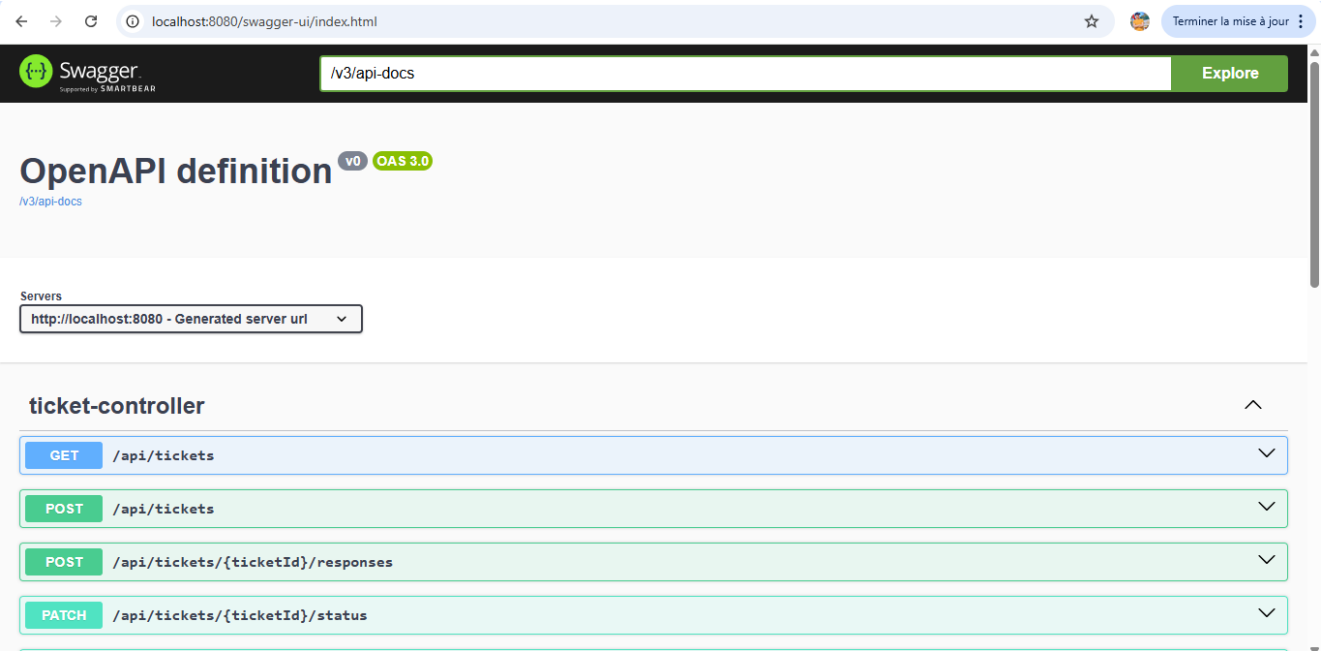
Architecture globale — DevOps Helpdesk API



Flux: Client → API ↔ DB | Repo → CI/CD → GHCR → Deploy

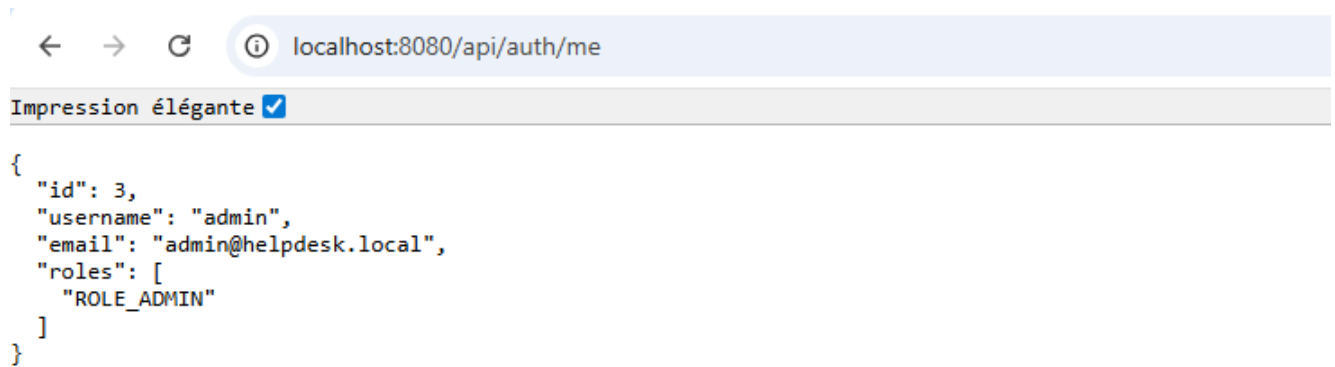
Capture 2: Documentation API via Swagger UI

02_swagger_home.png



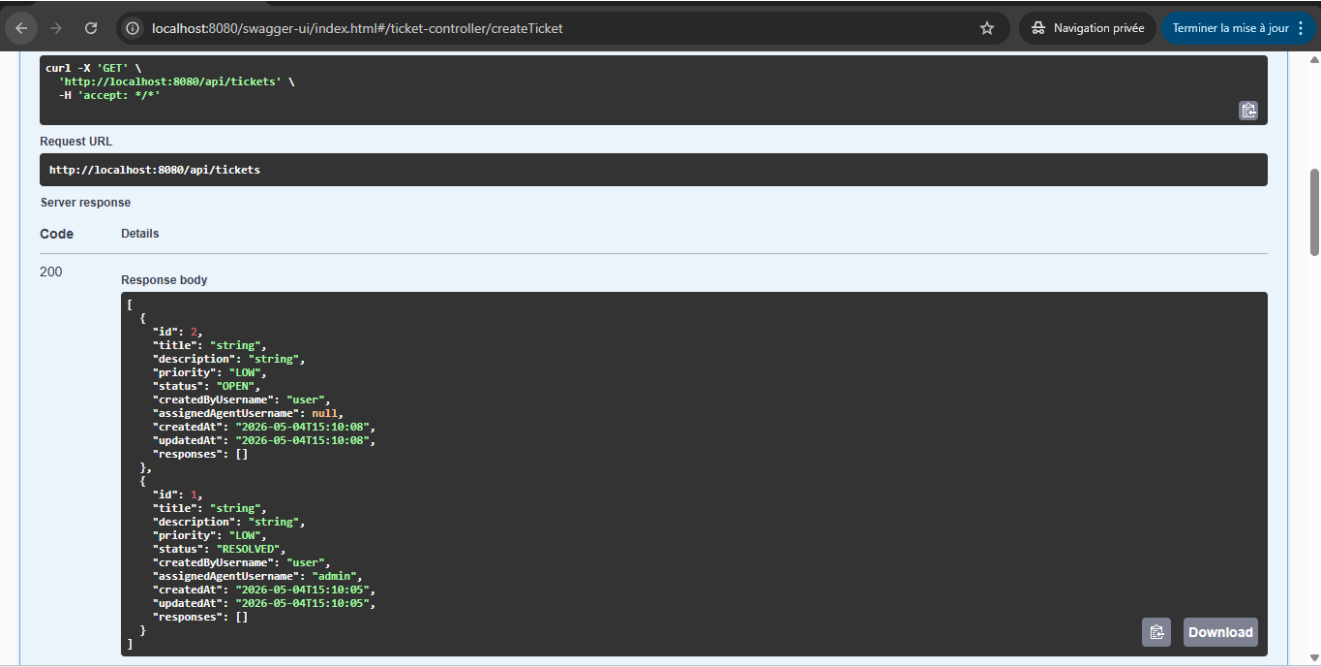
Capture 3: Test endpoint /api/auth/me (authentication OK)

03_auth_me_success.png



Capture 4: Création d'un ticket support (POST /api/tickets)

04_create_ticket.png



Capture 5: Mise à jour ticket: assignation/statut

05_update_status_or_assign.png

The screenshot shows the Swagger UI interface in a web browser. The browser's address bar displays the URL `localhost:8080/swagger-ui/index.html#/ticket-controller/updateStatus`. The interface is divided into several sections:

- Curl:** A text box containing a cURL command for a PATCH request to `http://localhost:8080/api/tickets/1/status` with headers `accept: */*` and `Content-Type: application/json`, and a JSON body `{ "status": "OPEN" }`.
- Request URL:** A text box showing the URL `http://localhost:8080/api/tickets/1/status`.
- Server response:** A section with tabs for **Code** and **Details**. The **Code** tab is selected, showing a **200** status code.
- Response body:** A text box displaying the JSON response body, which includes ticket details and a list of responses.

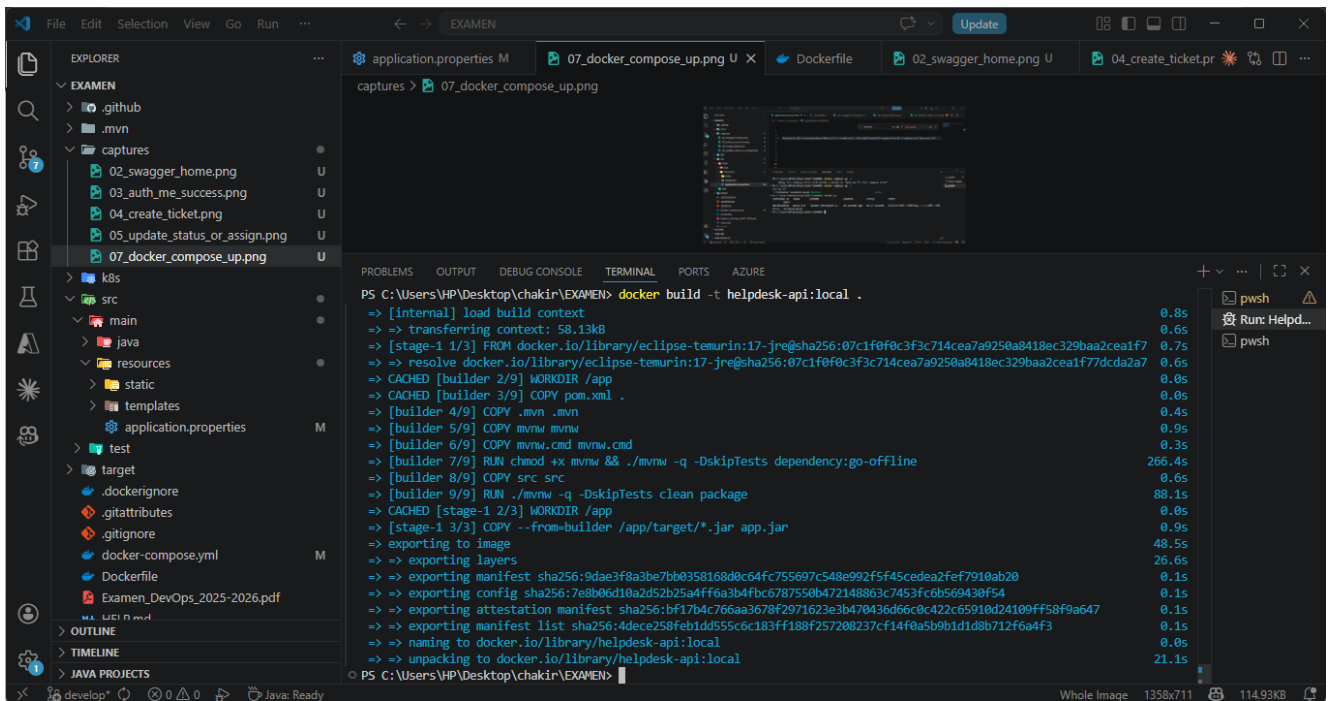
The JSON response body is as follows:

```
{
  "id": 1,
  "title": "string",
  "description": "string",
  "priority": "LOW",
  "status": "OPEN",
  "createdByUsername": "user",
  "assignedAgentUsername": "admin",
  "createdAt": "2026-05-04T15:10:05",
  "updatedAt": "2026-05-04T16:48:31.5715049",
  "responses": [
    {
      "id": 1,
      "message": "string",
      "authorUsername": "agent",
      "createdAt": "2026-05-04T16:47:48"
    }
  ]
}
```

At the bottom of the Swagger UI, there is a **Download** button. The browser's taskbar at the bottom shows the system clock as 16:48 on 04/05/2026.

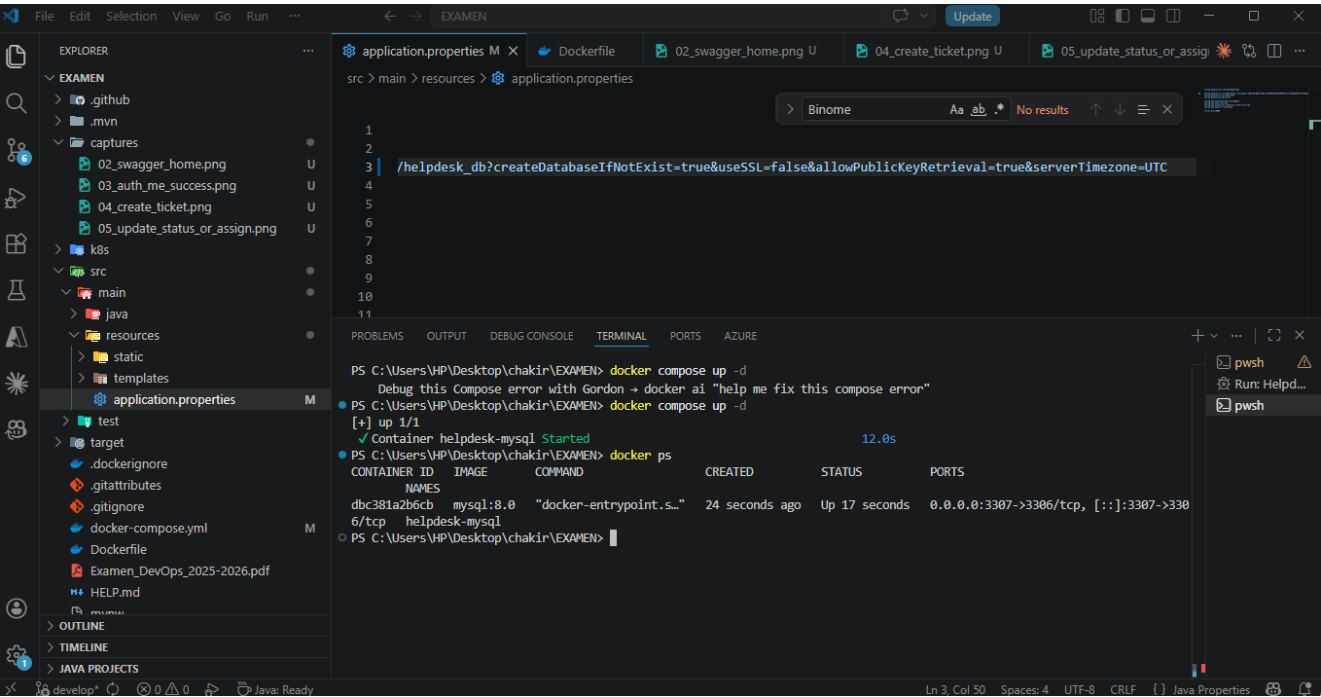
Capture 6: Build image Docker de l'application

06_docker_build.png



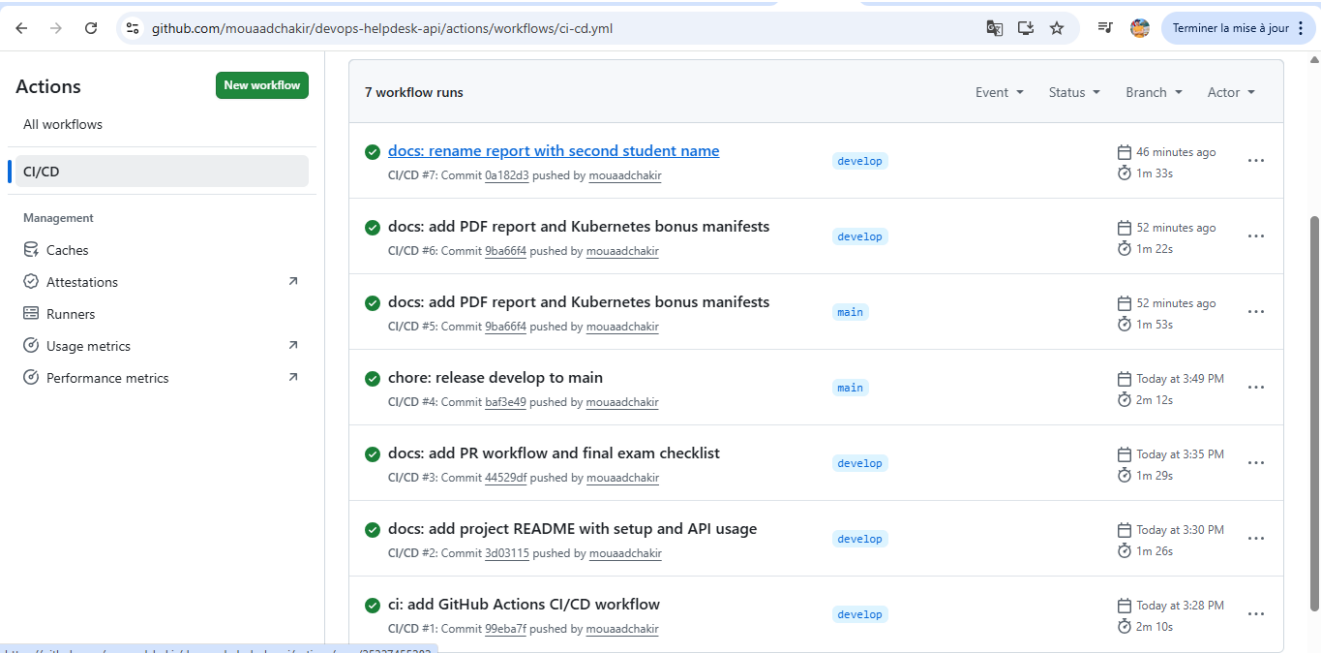
Capture 7: Exécution locale via Docker Compose

07_docker_compose_up.png



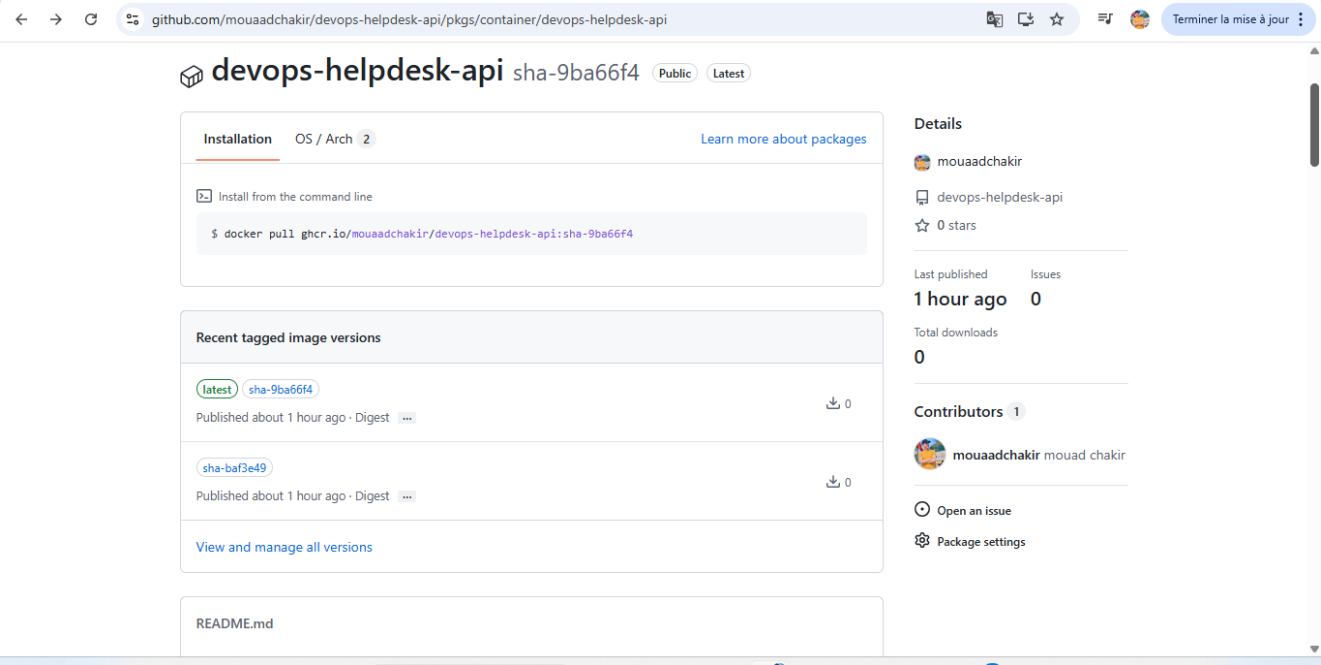
Capture 8: Pipeline GitHub Actions au vert

08_ci_pipeline_green.png



Capture 9: Image publiée sur GHCR

09_ghcr_image.png



Capture 10: Workflow Git (branches/PR)

10_git_pr_or_branches.png

← → ↺

github.com/mouaadchakir/devops-helpdesk-api/tree/develop

Terminer la mise à jour ⋮

develop 2 Branches 0 Tags

Go to file T Add file <> Code

This branch is 1 commit ahead of main. Contribute

docs: rename report with second student name ✓

0a182d3 · 51 minutes ago 13 Commits

.github	docs: add PR workflow and final exam checklist	1 hour ago
.mvn/wrapper	chore: setup Spring Boot and MySQL config	3 days ago
k8s	docs: add PDF report and Kubernetes bonus manifests	1 hour ago
src	feat: implement auth and ticket workflow APIs	3 days ago
.dockerignore	chore: add Dockerfile for API image	3 days ago
.gitattributes	chore: setup Spring Boot and MySQL config	3 days ago
.gitignore	chore: setup Spring Boot and MySQL config	3 days ago
Dockerfile	chore: add Dockerfile for API image	3 days ago
Examen_DevOps_2025-2026.pdf	chore: initial commit	3 days ago
README.md	docs: add PDF report and Kubernetes bonus manifests	1 hour ago
Rapport_DevOps_MouaadChakir_SouadChakir....	docs: rename report with second student name	51 minutes ago

Systeme de gestion de tickets : un utilisateur ouvre un ticket avec une description et une priorité, un agent y répond, le statut évolue (Ouvert / En cours / Résolu / Fermé). Filtres et statistiques par agent

Readme Activity 0 stars 0 watching 0 forks

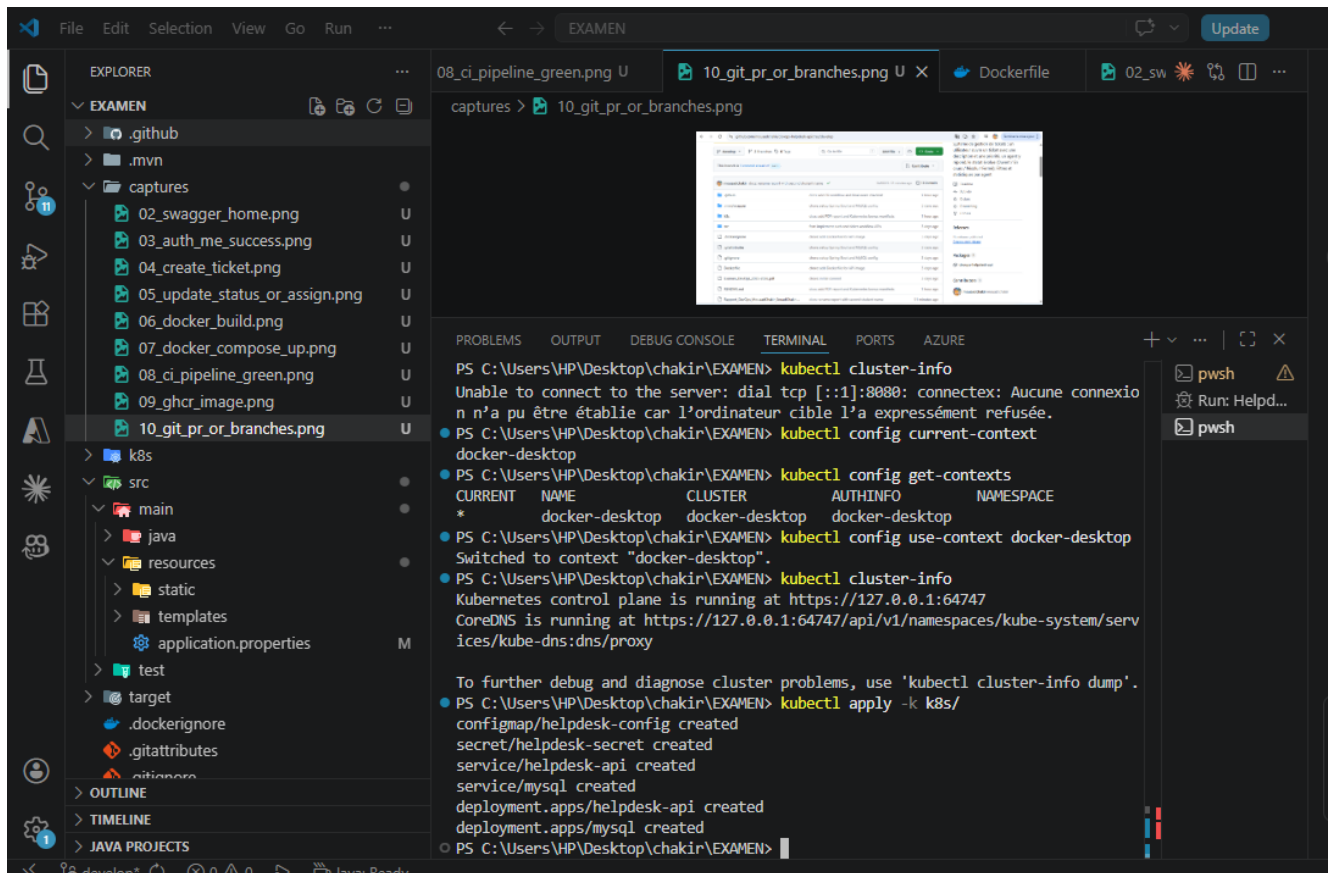
ReleasesNo releases publishedCreate a new release

Packages 1devops-helpdesk-api

Contributors 1mouaadchakir mouad chakir

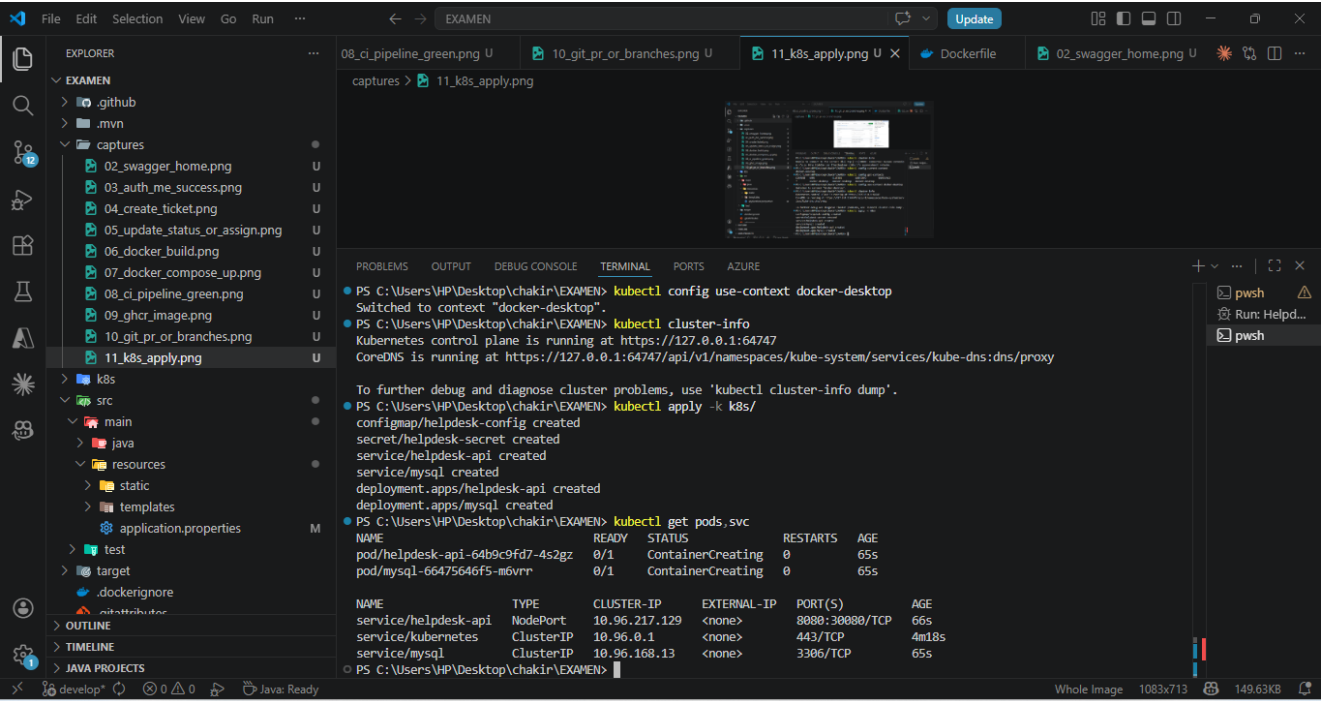
Capture 11: Déploiement Kubernetes (kubectl apply -k k8s/)

11_k8s_apply.png



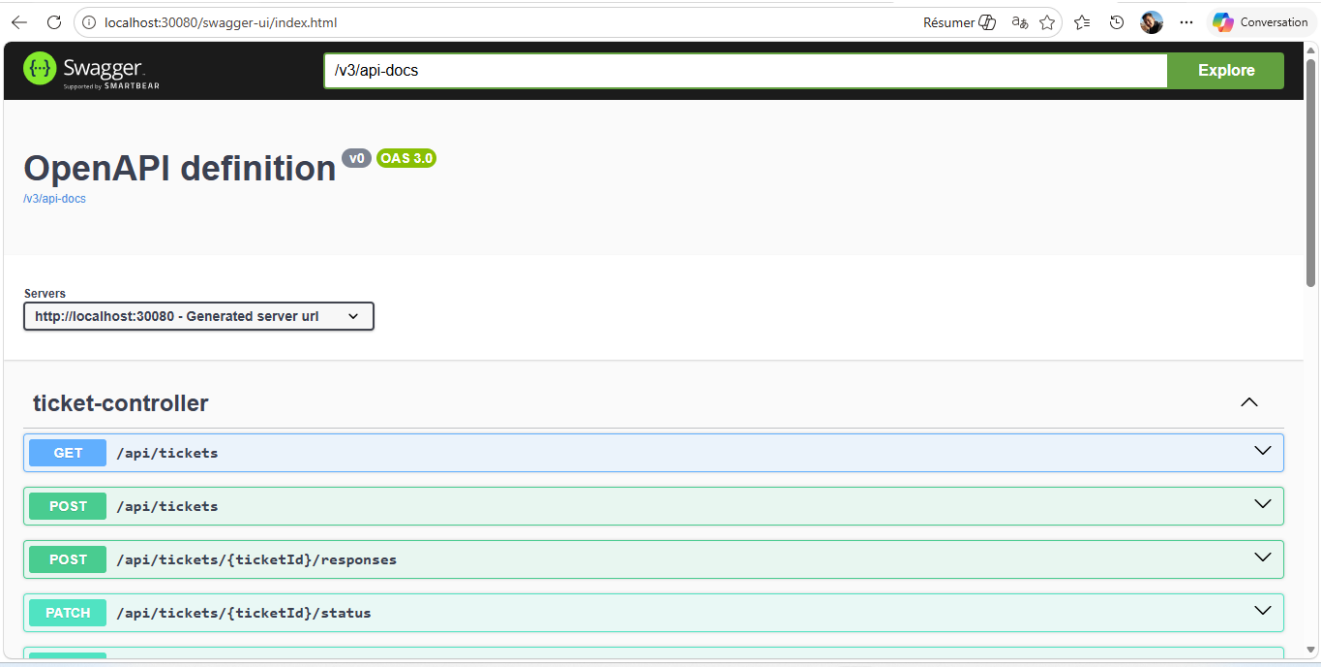
Capture 12: État du cluster (pods/services)

12_k8s_get_pods_svc.png



Capture 13: Accès Swagger via Kubernetes

13_k8s_swagger_access.png



6. Conclusion

Le projet répond aux exigences principales du module DevOps: application fonctionnelle, sécurité par rôles, pipeline CI/CD opérationnel, conteneurisation et documentation complète.

L'ajout du bonus Kubernetes démontre une capacité d'industrialisation supplémentaire et une préparation à des environnements d'exécution orchestrés.

Perspectives: migration vers JWT, tests d'intégration plus avancés, observabilité (metrics/logs/traces), et déploiement cloud managé.